High-Level Synthesis with Ant Colony Optimization Scheduling

Nnaemeka Nnachi-Egwu
Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany
nnaemeka.nnachi-egwu@stud.hshl.de

***Abstract*—High-level synthesis (HLS) transforms a behavioural hardware description into a register-transfer-level (RTL) netlist. Scheduling, the assignment of operations to clock cycles, is a central and NP-complete sub-task of this process. Classical Force-Directed Scheduling (FDS) provides near-optimal results in polynomial time, but its predecessor-successor force term carries $O(n^3)$ complexity and its single-pass nature cannot incorporate feedback from downstream synthesis steps. This paper presents a critical analysis and re-evaluation of an Ant Colony Optimization (ACO) approach to HLS scheduling proposed by Kopuri and Mansouri [1]. Their method replaces the PS-force term with two experience matrices representing per-cycle and cross-cycle learning from complete synthesis evaluations. The resulting modified force equation is computed in linear time. Benchmarks on four DSP circuits show that the ACO-enhanced scheduler yields costs equal to or lower than FDS in all tested configurations. This paper details the algorithm, reproduces and analyses the key equations, identifies a sign-convention inconsistency in the published formulation, implements the algorithm in C++17, and evaluates its suitability for embedded systems applications.**

***Index Terms*—high-level synthesis, scheduling, ant colony optimization, force-directed scheduling, register-transfer level, embedded systems**

I. Introduction

High-level synthesis automates the conversion of a behavioural hardware description, typically in a hardware description language or a C-based specification, into a structural netlist at the register-transfer level. The synthesised design must satisfy a set of architectural constraints while minimising hardware cost. Among the sub-tasks of HLS, scheduling is particularly critical: it determines which operation executes in which clock cycle, directly shaping the number of functional units required and therefore the silicon area and power consumption of the resulting datapath [1].

Scheduling under resource constraints or latency constraints is NP-complete in general [2]. Several heuristic approaches have been proposed, ranging from list scheduling with priority functions to ILP formulations. Force-Directed Scheduling (FDS), introduced by Paulin and Knight in 1987, has become widely used because it minimises concurrency peaks directly and delivers near-optimal results in polynomial time [2]. Its $O(n^3)$ time complexity, however, limits its scalability, and its single-pass nature prevents it from incorporating information about the downstream effects of scheduling decisions.

Meta-heuristic techniques, including genetic algorithms [4] and simulated evolution, have been applied to HLS scheduling to escape local optima through population-based or stochastic search. Ant Colony Optimization (ACO), proposed by Dorigo et al. in 1992 [3], is a multi-agent stochastic method inspired by pheromone-based communication in real ants. It has been applied to many NP-hard combinatorial problems. Kopuri and Mansouri [1] proposed an ACO-based enhancement to FDS in which the PS-force term is replaced by two experience matrices that accumulate information across multiple synthesis evaluations.

This paper provides a detailed technical analysis of the Kopuri-Mansouri approach. Section II reviews the FDS baseline. Section III presents the ACO-enhanced algorithm, with all equations transcribed from the primary source and verified against the original page images. Section IV identifies a sign-convention inconsistency in the published equations and analyses its implications. Section V evaluates the benchmark results and the method's suitability for embedded systems applications. Section VI describes a C++ implementation and VHDL RTL application. Section VII concludes.

II. Related Scheduling Approaches

HLS scheduling has attracted extensive research since the 1980s. List scheduling assigns operations to time-steps in topological order using a priority function; it runs in $O(N \log N)$ but the quality of the result depends entirely on the priority function, which is application-specific. ILP formulations guarantee optimal solutions but are exponential in the worst case and intractable for CDFGs with more than a few tens of operations. FDS [2] occupies a middle ground: polynomial time with near-optimal results.

Several researchers have applied meta-heuristic techniques to overcome the single-pass limitation of constructive heuristics. Heijligers et al. [4] used genetic algorithms and simulated evolution for combined scheduling and allocation; their approach considers multiple objectives but incurs high parameter sensitivity and slow convergence. Park and Kyung [5] proposed an iterative technique based on the Kernighan-Lin graph bisection approach, achieving near-optimal schedules at $O(N^2 \log N)$ complexity but with limited generality. Integrated scheduling frameworks such as InSyn [6] target DSP applications by coupling scheduling with application-specific optimisations.

At the intersection of ACO and HLS, Keinprasit and Chongstitvatana [7] proposed "dynamic ants" for HLS synthesis using a decision-path graph with dynamic niche sharing. Their approach does not close the full synthesis loop, and the cost model is less clearly defined than in [1]. The Kopuri- Mansouri work is distinguished by its direct use of post-synthesis cost as the ACO reinforcement signal and its grounding in the well-understood FDS framework.

The ACO paradigm itself has been extended far beyond the original Ant System (AS) of Dorigo [3]. The Ant Colony System (ACS) introduces local pheromone updates and explicit exploitation-exploration balance. Max-Min Ant Systems bound pheromone trails to prevent premature convergence. None of these variants are adopted by Kopuri and Mansouri, whose trail update mechanism is a novel custom formulation for the assignment problem structure of HLS scheduling.

III. Force-Directed Scheduling Baseline

FDS operates on a Control-Data Flow Graph (CDFG) in which nodes are operations and directed edges represent data dependencies. Under a fixed latency constraint λ, ASAP and ALAP scheduling determine for each operation i a *time frame* $[t_i^S,\, t_i^L]$ within which it must be scheduled. The *type distribution* $q_k(l)$ expresses the expected concurrency of operation type k at time-step l, computed by summing the uniform scheduling probability $p_i(m) = 1/(t_i^L - t_i^S + 1)$ of each unscheduled operation of type k.

The *self-force* measures the direct concurrency impact of scheduling operation i at time-step l:

$$S_{il} = \sum_{m=t_i^S}^{t_i^L} q_k(m)\big(\delta_{lm} - p_i(m)\big), \tag{1}$$

where δ_{lm} is the Kronecker delta. Positive self-force indicates increased concurrency pressure; negative self-force indicates relief.

When scheduling i at l constrains a predecessor or successor, its time frame changes from $[t_i^S, t_i^L]$ to $[\tilde{t}_i^S, \tilde{t}_i^L]$. The *PS-force* captures this secondary effect:

$$PS_{il} = \frac{1}{\bar{\mu}_i + 1} \sum_{m=\tilde{t}_i^S}^{\tilde{t}_i^L} q_k(m) - \frac{1}{\mu_i + 1} \sum_{m=t_i^S}^{t_i^L} q_k(m), \tag{2}$$

where μ_i and $\bar{\mu}_i$ are the mobility (time-frame width minus one) before and after the scheduling event. At each step, FDS selects the operation and time-step with the minimum total force $S_{il} + \sum PS_{il}$. The PS-force computation visits all predecessors and successors, yielding $O(n^3)$ overall complexity.

IV. ACO-Enhanced Scheduling

A. Overview

Kopuri and Mansouri replace the PS-force term in FDS with two matrices encoding experience from previous synthesis iterations. The per-cycle experience matrix $M^n(N, \lambda)$ stores pheromone trails deposited by individual ants during iteration n. The global experience matrix $G(N, \lambda)$ accumulates trails across all cycles. Both are initialised to zero.

The outer algorithm, Ant_sched, runs n_{iter} synthesis cycles. In each cycle, $n_{\text{ants}} = \lfloor \eta N \rfloor$ ants are initialised with the operations having the least self-forces as their first scheduled operations. Each ant independently executes procedure A_FDS to produce a complete schedule, which is then passed through clique-partitioning resource allocation and Left-Edge register allocation to yield a synthesised design with a computable hardware cost. The trail matrix is updated from the cost result, and the global matrix is updated after all ants in the cycle have run. Table I lists all parameters with their empirically determined values.

TABLE I
ALGORITHM PARAMETERS [1]

Parameter	Meaning	Value
η	Ant fraction / candidate fraction	0.5
α	Self-force weight	1
β	Per-cycle trail weight	2
γ	Global trail weight	2
ρ	Evaporation factor	0.35–0.50
n_{iter}	Max. iterations	20
n_{ants}	Ants per iteration	20

B. Trail Update

After ant m in cycle n produces schedule s with cost $\text{cost}(s)$, the per-cycle trail matrix is updated as follows:

$$M_{ij}^n = M_{ij}^n + \frac{\text{cost}(s) - \text{cost}(s^o)}{\text{cost}(s^o)}, \quad \forall (i,j) \in s, \tag{3}$$

where s^o is the best schedule found so far. If $\text{cost}(s) < \text{cost}(s^o)$, the schedule s replaces s^o.

C. Global Experience Update

After all ants in cycle n have run, the global matrix is updated with evaporation:

$$G_{ij}^{n+1} = \rho G_{ij}^n + M_{ij}^n, \quad \forall (i,j). \tag{4}$$

The evaporation factor $\rho \in (0, 1)$ causes older trail information to decay, preventing premature convergence. Values of ρ between 0.35 and 0.50 produced the best empirical results [1].

D. Modified Force Equation

Procedure A_FDS uses the following force equation to evaluate scheduling operation i at time-step l:

$$F_{il} = \alpha \frac{S_{il}}{\sqrt{\dfrac{1}{N\lambda} \displaystyle\sum_{(i,j)=(1,1)}^{N,\lambda} S_{ij}}} - \beta M_{il}^n - \gamma G_{il}^n. \tag{5}$$

The first term normalises the self-force by its root-mean-square over all (i, j) pairs, ensuring that the self-force and experience terms are commensurate. The second and third terms subtract current-cycle and cross-cycle experience, respectively. All three terms are computable in $O(N\lambda)$ time per scheduling step, as opposed to $O(n^3)$ for FDS [1]. Constants α, β, and γ are determined empirically.

V. Sign-Convention Analysis

Close reading of [1] reveals a sign-convention inconsistency between the prose description and the published equations.

A. Mathematical Derivation of the Inconsistency

When schedule s is better than s^o, i.e., $\text{cost}(s) < \text{cost}(s^o)$, the numerator of the increment in Eq. (3) is negative, so M_{ij}^n decreases for the operations (i, j) used in the improved schedule. In Eq. (5), the term $-\beta M_{il}^n$ then increases F_{il}, since $\beta > 0$ and M_{il}^n has become more negative. A_FDS selects the operation and time-step with *minimum* force. Consequently, scheduling decisions that contributed to improved synthesis results receive a higher force value, making them *less* likely to be re-selected in the next ant's traversal.

Numerical example. Suppose $\text{cost}(s^o) = 1000$ and $\text{cost}(s) = 900$ (a 10% improvement). The trail increment from Eq. (3) is $(900 - 1000)/1000 = -0.10$. With $\beta = 2$, the contribution to F_{il} is $-2 \times (-0.10) = +0.20$, raising the force. If conversely the schedule worsens to $\text{cost}(s) = 1100$, the increment is $+0.10$ and the contribution to F_{il} is -0.20, reducing the force and making the worsening decision *more* attractive in the next step. The pheromone mechanism reinforces bad decisions and suppresses good ones.

B. Possible Interpretations

This contradicts the paper's prose: "The trail laid by an ant is positive or negative depending on whether improved synthesis results are obtained or not." Mathematically, the trail increment is always negative for an improvement. Two consistent corrections are possible:

1) **Negate Eq.** (3): change the increment to $-(\text{cost}(s) - \text{cost}(s^o))/\text{cost}(s^o)$, yielding a positive increment for improvement.
2) **Change signs in Eq.** (5): write $+\beta M_{il}^n + \gamma G_{il}^n$ and select the *maximum* force, equivalently inverting the selection criterion.

Despite this inconsistency, the benchmark results show improvement over FDS, suggesting that the anti-reinforcement may inadvertently function as a diversification mechanism, preventing all ants from converging on the same FDS-like schedule. The normalised self-force term dominates Eq. (5) and provides the primary scheduling guidance; the trail terms perturb the search around the FDS solution. The C++ implementation described in Section VI confirms that correcting the sign produces faster convergence and, in some cases, lower final costs. Without the authors' source code, the intended behaviour cannot be definitively determined.

VI. Benchmark Evaluation and Embedded Systems Discussion

A. Benchmark Results

The algorithm was evaluated on four DSP benchmarks: Differential Equation, Elliptical Filter, Discrete Cosine Transform 1 (DCT1), and DCT2 [1]. Hardware costs were computed using the unit costs in Table II.

TABLE II
Hardware unit costs [1]

Hardware Unit	Cost
Single Cycle Multiplier	250
Single Cycle Adder/Subtractor/Comparator	50
Register	15
2-input Mux	15

Table III reproduces the paper's Table 4. A_FDS matches or improves FDS in all eight benchmark-latency configurations. The largest improvements occur for DCT2 at latency 13 (-10.3%) and DCT1 at latency 14 (-5.9%). No improvement is observed for the Differential Equation benchmark at either latency. All experiments ran in "a few seconds" on a Pentium 3 / 800 MHz machine with 320 MB RAM.

TABLE III
Cost of best solution: FDS vs. A_FDS [1]

Benchmark	Latency	FDS	A_FDS
Diff. Eq.	4	875	875
Diff. Eq.	5	875	875
Elliptical Filter	10	825	775
Elliptical Filter	12	775	775
DCT 1	12	1795	1730
DCT 1	14	1795	1690
DCT 2	13	1210	1085
DCT 2	14	1525	1390

B. Critical Assessment of Results

The experimental evaluation has several limitations. Each data point appears to represent a single run; ACO is stochastic and multiple runs with variance reporting would be necessary to establish statistical significance. No comparison is made against other meta-heuristics such as simulated annealing or tabu search, which are natural baselines for iterative HLS scheduling. The benchmarks are exclusively arithmetic-intensive DSP kernels with regular data dependency graphs, which may favour the self-force-dominated scheduling heuristic. The zero improvement on the Differential Equation benchmark, which has a small CDFG, suggests that the ACO trail does not accumulate useful signal for small problem instances.

C. Embedded Systems Suitability

A_FDS is well suited to ASIC datapath synthesis for DSP applications such as filter cores, transform engines, and linear algebra accelerators, where the count-based cost model and latency-constrained scheduling model are appropriate. The algorithm integrates naturally into a standard HLS toolchain, as it requires no modification to the resource allocation or register allocation steps.

The method is less appropriate for FPGA targets, where the cost metric must account for LUT packing and routing congestion rather than unit counts. It is also unsuitable for control-flow-heavy designs, memory- bandwidth-bound applications, or any scenario requiring formal timing guarantees, since the

stochastic scheduler cannot provide worst-case execution time bounds. Real-time embedded systems requiring deterministic response require a deterministic scheduler.

VII. Implementation

A. C++ Implementation

Both FDS and A_FDS were implemented in C++17 as a reference implementation accompanying this seminar work. The implementation follows the Ant_sched and A_FDS pseudocode from [1] exactly as printed, including the as-published sign convention in Eq. (3). Key design decisions and observations:

- The CDFG is represented as an adjacency list with operation type, ASAP, and ALAP annotations. ASAP and ALAP are computed by standard topological forward and backward traversal.
- Trail matrices M^n and G are `std::vector<double>` objects of size $N \times \lambda$. Memory consumption is negligible for benchmarks with $N \leq 50$ and $\lambda \leq 20$.
- The type distribution $q_k(l)$ is recomputed from the current partial schedule at each scheduling step. Scheduled operations contribute a point mass at their assigned time-step; unscheduled operations contribute $p_i(m) = 1/(t_i^L - t_i^S + 1)$ over their remaining time frame.
- The RMS normalisation denominator in Eq. (5) is computed once per A_FDS invocation before the scheduling loop begins and reused at each step.
- The clique partitioning for resource allocation uses a greedy compatible-operation interval colouring: operations of the same type that execute in non-overlapping time intervals are assigned to the same functional unit instance.
- Register allocation uses the Left-Edge algorithm on variable live ranges derived from the schedule; each live range is a consecutive interval of time-steps during which a value must be stored. Overlapping intervals require separate registers.
- Multiplexer cost is computed from the number of distinct values written to each functional unit input across all time-steps.
- Running the implementation on Differential Equation, Elliptical Filter, DCT1, and DCT2 CDFGs confirms the result pattern of Table III: A_FDS costs are equal to or lower than FDS in all cases. The zero-improvement result on the Differential Equation benchmark is reproduced.

The implementation makes the sign-convention issue directly observable: setting the increment sign in Eq. (3) to positive (the corrected interpretation) causes A_FDS to converge faster and to lower costs on DCT benchmarks, while the as-printed negative sign still yields improvement but with slower convergence. This strongly suggests that the original intent was the positive sign and that Eq. (3) contains a typographic error.

B. Algorithm Complexity Summary

Table IV compares the per-iteration complexity of FDS and A_FDS, and the total work across the outer scheduling loop.

For the benchmark parameter values ($n_{\text{iter}} = n_{\text{ants}} = 20$, $N \leq 50$, $\lambda \leq 20$), the total work is dominated by the downstream synthesis tools (clique partitioning at $O(N^2)$ and Left-Edge at $O(N \log N)$) called 400 times, which is approximately $400 \times O(N^2) = 400 \times 2500 = 10^6$ operations for $N = 50$, consistent with the "few seconds" runtime claim.

TABLE IV
Complexity comparison

Algorithm	Per scheduling step	Total
FDS	$O(N^3)$	$O(N^4)$
A_FDS (one ant, one iteration)	$O(N\lambda)$	$O(N^2\lambda)$
Ant_sched full	$O(N\lambda)$ per step	$O(n_{\text{iter}} \cdot n_{\text{ants}} \cdot N^2\lambda)$

C. VHDL RTL Application

The scheduler output defines a datapath netlist: for each time-step, which operation executes on which functional unit, and which values must be stored in registers between steps. A VHDL entity is generated from this schedule, with separate processes for each functional unit type (multipliers, adders) and a clocked register stage. The generated VHDL is synthesisable and structurally corresponds to the RTL level of the HLS output as described in [1]. A sample VHDL entity for a two-multiplier, two-adder datapath operating under latency λ is provided in the accompanying `implementation/vhdl/` directory.

VIII. Conclusion

This paper has presented a detailed analysis of the ACO-enhanced HLS scheduler proposed by Kopuri and Mansouri [1]. The algorithm modifies Force-Directed Scheduling by replacing the expensive PS-force term with two lightweight experience matrices updated from complete post-synthesis evaluations, reducing the per-step complexity from $O(n^3)$ to $O(N\lambda)$.

Benchmark results confirm that A_FDS matches or improves FDS across all tested DSP circuits within 20 iterations, with the most significant gains (up to 10.3% cost reduction) on the DCT benchmarks. A critical finding of this analysis is a sign-convention inconsistency between the published Eq. (3) and the paper's prose: as formulated, improved scheduling decisions receive a higher force value, anti-reinforcing them rather than reinforcing them. The benchmarks nevertheless show improvement, which may be attributable to an inadvertent diversification effect.

The method is most appropriate for ASIC datapath synthesis of arithmetic- intensive DSP circuits under latency constraints. Extensions to FPGA cost models, multi-objective optimisation, and parallel ant evaluation are identified as productive directions for future work. A corrected and statistically validated re-evaluation of the sign-convention alternatives would be a natural starting point for such work.

Acknowledgements

I thank Prof. Dr. Achim Rettberg for supervising this seminar work and for providing the reference materials and topic. AI usage during this work is documented in the accompanying AI usage protocol files as required by the seminar protocol.

REFERENCES

[1] S. Kopuri and N. Mansouri, "Enhancing scheduling solutions through ant colony optimization," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 2004, pp. V-257–V-260.

[2] P. G. Paulin and J. P. Knight, "Force-directed scheduling in automatic data path synthesis," in *Proc. DAC*, 1987, pp. 195–202.

[3] M. Dorigo, *Optimization, Learning and Natural Algorithms*, Ph.D. thesis, Politecnico di Milano, IT, 1992.

[4] M. G. M. Heijligers, L. J. M. Cluitmans, and J. A. G. Jess, "High level synthesis, scheduling and allocation using genetic algorithms," in *Proc. ASP-DAC*, 1995, pp. 61–66.

[5] I. C. Park and C. M. Kyung, "Fast and near optimal scheduling for data path synthesis," in *Proc. DAC*, 1991, pp. 650–685.

[6] A. Sharma and R. Jain, "InSyn: Integrated scheduling for DSP applications," in *Proc. Int. Symp. High-Level Synthesis*, 1994, pp. 96–103.

[7] R. Keinprasit and P. Chongstitvatana, "High level synthesis by dynamic ants," *Int. J. Intell. Syst.*, 2003.